Time and Space complexity

09 April 2026 08:19

```
for (int i=0 ; i<n ; i++){
    for (j=0 → n){
        for(k=0 → n){
            Sout(*);
        }
    }
}
```

T.C = $O(n^3)$ //

logarithmic time Complexity

$O(\log n)$

```
int n = 64;
while(n>0){
    Sout(n);
    n = n/2;
}
```

n=64 → 6

int n = 128 → 8

n=64 → 7

64
32
16
8
4
2
1

$2^5 = 32$
64
128

$\frac{128}{2^7} = 1$

- k times

$\frac{n}{2^{k-1}} = 1 \Rightarrow n = 2^{k-1}$

take both side $\log_2$

$\log_2 n = \log_2 2^{(k-1)} \Rightarrow \log_2 n = k-1$ $\log_2 2 \to 1$

$k = \log_2 n + 1$

n

$\boxed{k = \log_2 n}$

n = 5;

```
int star = 0;
for (int i=1; i<=n; i++){
    if (i!=1 && i%2!=0){
        star++;
    }
    for(int j=1; j<=star; j++){
        Sout("*");
    }
    Sout("\n");
}
```

row
1 → *
3 → * * *
5 → * * * * *
7 → * * * * * * *

mid
2 3 2

i=1 → 1
i=2 → 2 3 2
i=3 → 3 2 1 2 3
i=4 → 4 3 2 1 2 3 4

```
for (int i=1 → n) {
    for( int j=1 → n-i) Sout(" ")
    int count = i;
    for(int j=1 → 2*i-1) {
        Sout (count);
        if (j>i) count++;
        else count--;
    }
}
```

1 ..... 2i-1

$\boxed{2n-1}$

$\boxed{2n+1}$

T.C ?

maximum case

$n-1$ $2*n-1$

$O(n(n-1) + n(2n-1))$

$O(n^2 - n + 2n^2 - n)$

$O(3n^2 - 2n)$ → neglect

const

→ $\boxed{O(n^2)}$

n=64
→ for(int i=0; i<n; i++) → n times

(. $O(n)$.

↳ for(int i=0; i<n; i++)

↳ $O(n)$.

↳ $\boxed{n=64;}$

while (n>0) {

k times { Sout(n);

n = n/2;

}

→ 7

$n = 128/267/124$

$\frac{n}{2^{k-1}} = 1$

$\frac{64}{2^{7-1}}$ $\frac{64}{2^6} = 1$

$n = 2^{k-1}$

take $\log_2$ on both sides.

$\log_2 n = \log_2 (2)^{k-1}$

$\log_2 n = k-1 \log_2 2$

base = 2

$\log_2 n = k-1$

$\log_2 n + 1 = k$

$O(k) \rightarrow O(\log_2 n + 1)$

$O(\log_2 n) \approx O(\log n)$

Properties

$\log_n (n^3)^{10}$

$= 10 \times \log_n n^3$

$\log_2 16$

$= 4$

$\log_2 4$ $2^2 = 4$

$= 2$

constraints

< 10

< [15 ... 18]

n < 100

n < 400

n <= 10^4

n <= 10^5

→ n <= 10^8

n <= 10^9

↳ 2×10^8 (x).

Time Complexity

$O(n!)$

$O(2^n)$

$O(n^4)$

$O(n^3)$

$\boxed{O(n^2)}$

$O(n \log n)$ → later

→ $O(n)$

$O(\log n)$

$O(1)$

$\boxed{n = 10^4}$

$O(n^2)$

→ $O(n \log n)$

→ $O(n)$

→ $O(\log n)$

n is what we are traversing.

Cutoff → 10^8 → (x)

Constraint

$n <= 10^4$ → $O(n)$ → 10^4 times

$n = 1$ ✗

$n = 10^4$

Main Imp.

$10^4 <= 10^8$

$n <= 10^4$ → (n^2)

10^8

$10^8 <= 10^8$

$n <= 10^4$ n^3 → $10^{12} <= 10^8$ ✗

$n <= 10^9$ → $O(n)$ → 10^9

$10^9 <= 10^8$ ✗

$2^8 <= 10^8$

$\log n$ → $\log_2 10^9 = 9 \times \log_2 10$

$= 9 \times 3 \ldots = 27 \ldots$

$O(n \log n)$ → maximum

$n <= 10^5$

$O(n^2)$ ✗

$(10^5)^2$

$= 10^{10}$

$10^{10} <= 10^8$ ✗

$5 \times 3 \ldots$ $O(n)$ → 10^5

$10^5 <= 10^8$

$n \log n$ → $17 \ldots \times 10^5$

17×10^5

n log n

Space Complexity.

$O \rightarrow$ ✓

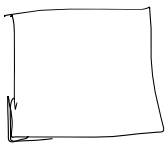
θ ✗

ω ✗

```
int a=1;
```

$\rightarrow 1$

Memory



$O(1)$; //Space Complexity

int a, b, c, d, e;

$O(5)$; → constant → $O(1)$.

int a, b, c, d ..... n variables.

$O(n)$

$O(n)$ → $f(x)$.

array

$O(n)$

[- - - - -]

int [] arr = new int [n]; → Space complexity

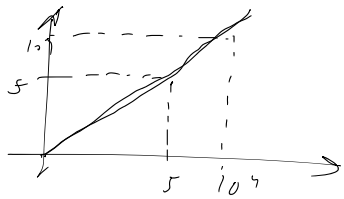
→ Asymptotic functions

→ // $O(n^1)$ → linear

$O(5)$ → time

$y = mx^1 + c$

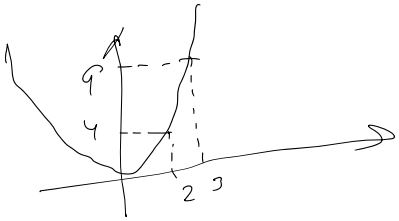
$n=5$



[[1,2,3], [3,2,1], [2,1,2]]

→ $O(n^2)$ ✓ → Quadratic

$n=2$

$n=2$.

4